

jq, ELK, Loki et Prometheus

Quatre outils, quatre catégories — comprendre leurs rôles avant de manipuler la stack d'observabilité.

jq

ELK

Loki

Prometheus

Une confusion courante à lever

Ces 4 outils ne jouent PAS dans la même catégorie



jq

Local

Outil CLI local — filtre du JSON sur votre machine, ne stocke rien.



ELK Stack

Centralisé

Elasticsearch + Logstash + Kibana — centralisation puissante avec full-text search.



Grafana Loki

Centralisé

Système de logs centralisé léger — indexe uniquement les labels, pas le contenu.



Prometheus

Métriques

Base de données de métriques (chiffres) — ne stocke PAS de logs.

jq — Le scalpel du JSON

Outil CLI ultra-léger pour analyser du JSON en local

Qu'est-ce que c'est ?

- Outil en ligne de commande (CLI) ultra-léger
- Ne stocke rien, ne centralise rien
- Filtre et transforme du JSON à la volée
- Idéal pour : analyser un fichier local ou en SSH sur un serveur

Quand l'utiliser ?

- ✓ Debug rapide sur votre poste
- ✓ Analyse d'un seul fichier de log
- ✓ Script CI/CD pour parser une réponse API JSON
- ✗ Inutilisable pour 50 serveurs en production

```
$ cat app.log | jq 'select(.level == "ERROR") |  
  .message'
```

```
"Database connection timeout on /checkout"
```

```
$ cat app.log | jq '  
select(.level == "ERROR") |  
{user:.user_id, msg:.message}'
```

```
{  
  "user": "user_789",  
  "msg": "Payment timeout"  
}
```

ELK Stack — Le poids lourd de la recherche

Elasticsearch · Logstash · Kibana



Le secret : Full-Text Indexing

Elasticsearch indexe chaque mot de chaque ligne de log.
Comme Google pour vos logs — recherche instantanée sur n'importe quel terme.

✓ Avantages

- Recherche full-text ultra-puissante
- Ecosystem très riche (Beats, APM)
- Grande communauté et documentation

✗ Inconvénients

- Très coûteux en RAM et stockage
- Configuration complexe
- Courbe d'apprentissage raide (Lucene/KQL)

Grafana Loki — Le Prometheus des logs

Léger, économique, pensé pour Kubernetes et Grafana

Applications

Logs JSON



Promtail / Alloy

Agent collecteur



Loki

Stockage compressé (S3)



Grafana

Visualisation LogQL

Le secret : index sur les labels uniquement

Loki n'indexe que les métadonnées (labels) : service="api", env="prod".

Le contenu du log est compressé et stocké brut — bien moins coûteux qu'ELK.

```
# LogQL — chercher des erreurs timeout
{service="api", level="ERROR"} |= "timeout"
```

```
# Rate d'erreurs par minute
rate({service="api"} | level="ERROR" [1m])
```

✓ Avantages

- Très faible consommation RAM/stockage
- Intégration native Grafana
- Parfait pour Kubernetes
- Open source, architecture simple

ELK vs Loki — L'approche d'indexation

La différence fondamentale entre les deux systèmes

ELK

Index full-text

Log JSON reçu :

```
"timestamp": "2026-06-14T22:00:00Z"  
"service": "payment-api"  
"level": "ERROR"  
"message": "Database timeout..."
```

↓ **Elasticsearch indexe :**

payment-api • ERROR • Database

timeout • 2026 • 22 • 00

→ Chaque mot est dans le dictionnaire !

Loki

Index labels seulement

Log JSON reçu :

```
"timestamp": "2026-06-14T22:00:00Z"  
"service": "payment-api"  
"level": "ERROR"  
"message": "Database timeout..."
```

↓ **Loki indexe SEULEMENT :**

service="payment-api" (label)

level="ERROR" (label)

→ Message = compressé en bloc brut

Prometheus — Le gardien des métriques

Des chiffres, pas du texte — complémentaire aux logs

Prometheus ne stocke PAS de logs.

Il scrappe périodiquement des endpoints /metrics exposant des chiffres :
→ CPU (85%), requêtes/s (200 req/s), taux d'erreurs (1.2%), latence P95 (340ms)...

Les 4 types de métriques Prometheus

Counter	Ne fait que monter (ex: total requêtes)	<code>http_requests_total</code>
Gauge	Monte et descend (ex: RAM libre)	<code>node_memory_MemFree_bytes</code>
Histogram	Distribution + quantiles calculés	<code>http_request_duration_seconds</code>
Summary	Quantiles pré-calculés côté app	<code>rpc_duration_seconds</code>

```
# PromQL — taux d'erreurs
sum(rate(
  http_requests_total{
    status=~"5.."
  }[5m]
)) /
sum(rate(
  http_requests_total[5m]
))
```

→ 0.012 (1.2% d'erreurs)

Duo Prometheus + Loki

Prometheus détecte le pic d'erreurs → vous basculez sur Loki pour lire les messages d'erreur correspondants avec les mêmes labels.

Un même log — quatre traitements différents

Trace JSON : erreur 500 sur /checkout

```
{ "timestamp": "2026-06-14T22:00:00Z", "service": "payment-api", "level": "ERROR",  
  "message": "Database connection timeout on /checkout", "duration_ms": 1500 }
```

Analyse locale

```
cat app.log | jq  
'select(.level=="ERROR")  
| .message'
```

Résultat : "Database connection timeout"

→ Debug ponctuel, 1 fichier

Indexation complète

Kibana → rechercher :
"timeout" ou "checkout"
(full-text, instantané)

Résultat : tous logs contenant ce mot

→ Prod, recherche libre

Labels + grep

```
{service="payment-api",  
  level="ERROR"}  
|= "timeout"
```

Résultat : flux filtré + scan texte

→ Kubernetes, faible coût

Métriques numériques

```
rate(http_requests_total  
{service="payment-api",  
  status="500"} [5m])
```

Résultat : 12 req/s en erreur — pas de texte

→ Alertes, dashboards tendances

Tableau comparatif

Synthèse des caractéristiques principales

Critère	jq	ELK	Loki	Prometheus
Nature	CLI local	Centralisé global	Centralisé léger	Métriques seulement
Ce qu'il indexe	Rien (à la volée)	Tout le contenu	Labels uniquement	Chiffres uniquement
Stockage	Aucun	Élevé (gros index)	Très faible (S3)	Faible (séries temp.)
Langage query	Syntaxe jq	KQL / Lucene	LogQL	PromQL
Full-text search	Oui (grep local)	Oui (ultra-rapide)	Oui (plus lent)	Non
Idéal pour	Debug ponctuel	Enterprise, budget+	Kubernetes / Cloud	Alertes, dashboards

⚠ La cardinalité — le piège de Loki

Notion clé à maîtriser pour ne pas tuer les performances

Qu'est-ce que la cardinalité ?

La cardinalité = le nombre de valeurs uniques possibles pour un label.

Plus elle est haute, plus Loki crée de chunks d'index → dégradation des performances.

✗ Labels à HAUTE cardinalité

- `label="user_id"` → millions de valeurs !
- `label="ip_address"` → illimité
- `label="request_id"` → unique par req
- `label="session_token"` → dangereux

✓ Labels à BASSE cardinalité

- `label="service"` → api, auth, db...
- `label="env"` → prod, staging
- `label="level"` → ERROR, WARN, INFO
- `label="job"` → app-sample, nginx

Règle : < 10 labels, < 1 000 valeurs uniques par label dans un flux de production.

Comment combiner ces outils ?

Les associations les plus courantes en production

Scenario 1 : Dev / Debug local

jq → grep local → vérification /metrics Prometheus

Outil idéal avant de pousser en staging.

Scenario 2 : Observabilité Production (notre stack TP)

Prometheus détecte un pic d'erreurs (alerte) → **Loki** affiche les logs correspondants → **Grafana** unifie le tout dans un dashboard

Même labels dans Prometheus et Loki → corrélation directe entre métriques et logs.

Scenario 3 — Enterprise : ELK + Prometheus + Grafana pour full-text search avancée + métriques + tableaux de bord unifiés.

Quel outil choisir ?

Guide de décision rapide

? Analyser un fichier de log sur ma machine ?

→ jq

? Production, budget cloud, recherche full-text libre ?

→ ELK

? Kubernetes, faible coût, déjà Grafana ?

→ Loki

? Surveiller des tendances, créer des alertes sur des chiffres ?

→ Prometheus

? Analyser les causes racines d'une alerte Prometheus ?

→ Loki + jq

Objectifs du TP

Ce que vous allez faire dans les 2 prochaines heures

01

ELK vs Loki + Promtail

Déployer la stack, configurer les labels, écrire des requêtes LogQL, visualiser dans Grafana.

02

Prometheus

Configurer les scrape targets, explorer /metrics, écrire des requêtes PromQL, créer une première règle d'alerte.

03

Incident simulé

Trigger l'incident de paiement (30% d'erreurs) et diagnostiquer la cause en combinant Loki + Prometheus.

À retenir

- jq = analyse locale rapide
- ELK = full-text search, infrastructure lourde
- Loki = labels + texte brut, léger, Kubernetes-native
- Prometheus = métriques numériques, pas de logs
- **En prod : Prometheus + Loki + Grafana = trio gagnant**

jq

ELK

Loki

Prom